CatBoost

학번	212STG15	이름	서혜정
학번	212STG33	이름	황예진

1. Introduction

CatBoost(Categorical Boosting)은 dataset에 permutation을 활용하는 ordered boosting 기법과 categorical feature를 다루는 알고리즘(ordered target statistics)을 담고 있는 gradient boosting 기반의 앙상블 모델링 방법이다.

2. Background

Datasets

먼저 m개의 features와 target $y_k \epsilon \mathbb{R}$로 이루어진 dataset $\mathcal{D} = \{(\boldsymbol{x}_k, y_k)\}_{k=1..n}, where\ \boldsymbol{x}_k = (x_k^1, \dots, x_k^m)$ 이 주어져 있다고 가정하자. x_k는 k 번째 관측치에 대한 m개 features value의 vector이다. $y_k \in \mathbb{R}$는 a target이고 binary or a numerical response 둘 다 가능하다.

Loss function

Expected loss $\mathcal{L}(F) := \mathbb{E}L(y, F(\boldsymbol{x}))$ 를 minimize하는 function $F: \mathbb{R}^m \to \mathbb{R}$ 을 학습하고자 한다. $(\boldsymbol{x}_k, y_k)$는 unknown distribution $P(\cdot,\cdot)$로부터 얻은 identically independent distributed한 training set이며, $(\boldsymbol{x}, y)$는 training set과는 독립적으로 P로부터 얻어진 test sample이라고 가정한다.

gradient decent algorithm

gradient boosting을 이해하려면 먼저 gradient decent algorithm를 알아야 한다. 이는 여러 종류의 문제에서 최적의 해법을 찾을 수 있는 일반적인 최적화 알고리즘이다.

a scalar function $f(x) with\ x \in \mathbb{R}$이 주어져 있을 때 $\min_x f(x)$ 를 하려고 한다. x 점에서의 $\frac{\partial f(x)}{\partial x}$는 gradient라고 불리며 함수를 증가시키는 방향을 의미한다. gradient descent method은 아래와 같다.

$$x^{(t+1)} = x^{(t)} - \alpha^{(t)} \nabla f\left(x^{(t)}\right) \quad t = 0,1,2,\ldots$$

목적함수의 기울기 $\nabla f\left(x^{(t)}\right)$ 와 반대방향으로 parameter x 를 업데이트하여 목적함수 $f\left(x^{(t)}\right)$ 를 최소화한다. 위 식에서 $\alpha^{(t)}$는 학습율(learning rate) 로 parameter step크기를 의미한다.

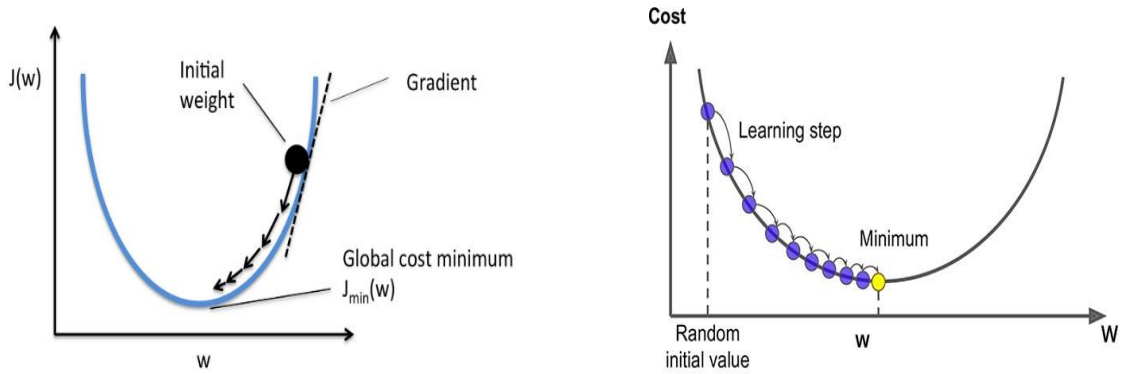

그림 1 gradient decent algorithm

Algorithm 1 Plain gradient descent

Input: starting point $x \in \mathbb{R}^n$, a function $\frac{\partial f(x)}{\partial x}$, stepsize α, tolerance θ

Output: some x hopefully minimizing f

1: **repeat**

2: $\quad x \leftarrow x - \alpha \frac{\partial f(x)}{\partial x}^{\top}$

3: **until** $\Delta x < \theta$ for 10 iterations in sequence

알고리즘 1 gradient descent algorithm

Ordinary least square (OLS)에 gradient decent를 적용하면 다음과 같다. Loss function이 squared error이면 $-g^t(x,y)$는 residual 값과 같다. 구체적인 과정은 다음과 같다.

Loss function of the ordinary least square: $\min L = \frac{1}{2}\sum_{i=1}^{n}\left(y_i - f(x_i)\right)^2$

gradient of the loss function: $\frac{\partial L}{\partial f(x_i)} = f(x_i) - y_i$

Residuals are the negative gradient of the loss function:

$$-g(x,y) = minus\ gradient = -\frac{\partial L}{\partial f(x_i)} = y_i - f(x_i) = residuals$$

gradient boosting

Gradient boosting 과정에서 만들어지는 각각의 트리는 이전 단계의 approximation을 반영한다. t번째 트리 F^t는 $F^t = F^{t-1} + \alpha h^t$라고 표현할 수 있다. 이 때 α는 step size(learning rate)이다.
함수 $h^t: \mathbb{R}^m \to \mathbb{R}$은 expected loss를 minimize하기 위해 family of function H에서 선택된 함수이다. 이를 식으로 표현하면 다음과 같다.

$$h^t = \underset{h \epsilon H}{\operatorname{argmin}} \mathcal{L}(F^{t-1} + h) = \underset{h \in H}{\operatorname{argmin}} \mathbb{E} L(y, F^{t-1}(\boldsymbol{x}) + h(x))$$

h_t 에 대한 최소화 문제는 $\mathcal{L}(F^{t-1} + h^t)$ 를 F^{t-1} 에 대해 미분하는 것이다. 이때 $h^t(x)$ 는 $-g^t(x,y) \quad where\ g^t(x,y) \coloneqq \frac{\partial L(y,s)}{\partial s}|_{s=F^{t-1}(x)}$ 로 근사할 수 있다. 그럼 위 식은 아래와 같이 쓰인다.

$$h^t = argmin_{h \in H} \mathbb{E}\left(-g^t(x,y) - h(x)\right)^2$$

Decision tree

Catboost의 개별 학습 트리는 각각 splitting attributes $\boldsymbol{a} = \mathbb{I}_{\{x^k > t\}}$에 따라 feature space를 몇 개의 region으로 분할한다. 이 때 attributes $\boldsymbol{a}$는 특정한 feature $\boldsymbol{x}^k$가 threshold를 t를 넘는지의 여부에 따른 binary variable이라고 할 수 있다. decision tree function h를 다음과 같이 표현할 수 있다.

$$h(x) = \sum_{j=1}^{J} b_j 1_{\{x \in R_j\}}$$

이 때 R_j는 트리의 leave로 분할된 region이고 b_j는 j번째 region의 y변수의 예측값을 뜻한다.

과거 gradient boosting 기법이 가지고 있었던 두가지의 공통적인 문제점은 prediction shift와 target leakage이다. Catboost에서는 categorical 변수 처리 과정에서 발생하는 Target leakage 문제를 ordered target statistics(ordered TS)로 해결한다. 또한 트리 형성 과정에서 발생하는 prediction shift 문제를 ordered boosting으로 해결한다. 3장에서 ordered TS를 알아보고 4장에서 ordered boosting을 알아보자.

3. Categorical Features

3.1 Related Work on Categorical

Categorical feature는 각각의 값들이 서로 비교할 수 없는 discrete value로 이루어진 변수이다. Categorical feature를 처리하는 가장 유명한 방법은 변수 내 각각의 category에 대한 binary feature를 만드는 one-hot encoding 방법이다.

		x^i		y
I_1	…	A	…	1
I_2	…	B	…	1
I_3	…	C	…	1
I_4	…	A	…	0
I_5	…	B	…	1
I_6	…	C	…	1
I_7	…	B	…	0
I_8	…	C	…	1
I_9	…	C	…	1
I_{10}	…	C	…	0

		$x^i(A)$	$x^i(B)$	$x^i(C)$		y
I_1	…	1	0	0	…	1
I_2	…	0	1	0	…	1
I_3	…	0	0	1	…	1
I_4	…	1	0	0	…	0
I_5	…	0	1	0	…	1
I_6	…	0	0	1	…	1
I_7	…	0	1	0	…	0
I_8	…	0	0	1	…	1
I_9	…	0	0	1	…	1
I_{10}	…	0	0	1	…	0

표 1 one hot encoding 예시

그러나 이 방법은 변수 내 category 수가 많을 경우 새로운 binary feature가 과도하게 많이 생성된다는 문제점이 있다. 이를 해결하기 위해 CatBoost 모델에서는 categorical feature의 각 category의 expected target value를 추정하는 target statistics(TS)에 따라 category들을 grouping하는 방법을 사용한다.

3.2 Target statistics

Greedy target statistics without smoothing

Greedy target statistics는 categorical feature의 각 category에 속하는 y들의 기대값과 같다. 즉 데이터의 i번째 feature가 categorical 변수일 때 k번째 sample의 target statistics는 $\hat{x}_k^i \approx \mathbb{E}(y \mid x^i = x_k^i)$이다. 각 category 하나에 하나의 numerical value가 지정되므로 category 개수가 많은 범주형 변수에 효과적이다. 그 과정에 대한 예시는 표 2와 같다.

		x^i		y
I_1	…	A	…	1
I_2	…	B	…	1
I_3	…	C	…	1
I_4	…	A	…	0
I_5	…	B	…	1
I_6	…	C	…	1
I_7	…	B	…	0
I_8	…	C	…	1
I_9	…	C	…	1
I_{10}	…	C	…	0

		$x^i(TS)$		y
I_1	…	0.50	…	1
I_2	…	0.67	…	1
I_3	…	0.80	…	1
I_4	…	0.50	…	0
I_5	…	0.67	…	1
I_6	…	0.80	…	1
I_7	…	0.67	…	0
I_8	…	0.80	…	1
I_9	…	0.80	…	1
I_{10}	…	0.80	…	0

표 2 greedy TS without smoothing

하지만 이 방법은 아래의 표3에서 알 수 있듯, noisy로 보이는 카테고리컬 변수는 경우에 수가 많이 있었던 카테고리컬 변수에 비해서 TS가 극단적인 값을 보일 수 있다. 아래 예시에서는 E의 TS가 1이다. 따라서 $greedy\ TS$에 $smoothing$을 적용한다.

	$y = 1$	$y = 0$	TS
A	10개	10개	0.5
B	40개	10개	0.8
C	10개	40개	0.2
D	25개	25개	0.5
E	1개	0개	1

표 3 a problem of greedy TS without smoothing

TS는 statistic을 어떤 dataset으로 계산하는지에 따라 greedy TS with smoothing과 Holdout TS로 나뉜다.

Greedy target statistics with smoothing

Greedy TS는 target statistics를 계산하는 가장 간단한 방법으로, 같은 카테고리에 속하는 y값에 평균을 취하여 $\hat{x}_k^i$를 얻는 방법이다. Target statistics $\hat{x}_k^i$을 계산한 식은 아래와 같다. 아래 식은 같은 카테고리에 속하는 데이터의 빈도수가 작은 추정값(nosiy 추정값)에 대하여 smoothing을 가능하게 한다. 아래식에서 p는 전체 데이터 셋에 대한 target value의 평균 값이다.

$$\hat{x}_k^i = \frac{\sum_{j=1}^{n} 1_{\{x_j^i = x_k^i\}} \cdot y_j + ap}{\sum_{j=1}^{n} 1_{\{x_j^i = x_k^i\}} + a}$$

이 때 $a > 0$는 조정 가능한 parameter이다. $\alpha = 0.1$일 때 실제 예시는 아래와 같다.

		x^i		y
I_1	…	A	…	1
I_2	…	B	…	1
I_3	…	C	…	1
I_4	…	A	…	0
I_5	…	B	…	1
I_6	…	C	…	1
I_7	…	B	…	0
I_8	…	C	…	1
I_9	…	C	…	1
I_{10}	…	C	…	0

		TS		y
I_1	…	$\frac{1+0.1\times0.7}{2+0.1} = 0.5095$	…	1
I_2	…	$\frac{2+0.1\times0.7}{3+0.1} = 0.6677$	…	1
I_3	…	$\frac{4+0.1\times0.7}{5+0.1} = 0.7980$	…	1
I_4	…	$\frac{1+0.1\times0.7}{2+0.1} = 0.5095$	…	0
I_5	…	$\frac{2+0.1\times0.7}{3+0.1} = 0.6677$	…	1
I_6	…	$\frac{4+0.1\times0.7}{5+0.1} = 0.7980$	…	1
I_7	…	$\frac{2+0.1\times0.7}{3+0.1} = 0.6677$	…	0
I_8	…	$\frac{4+0.1\times0.7}{5+0.1} = 0.7980$	…	1
I_9	…	$\frac{4+0.1\times0.7}{5+0.1} = 0.7980$	…	1
I_{10}	…	$\frac{4+0.1\times0.7}{5+0.1} = 0.7980$	…	0

표 4 greedy TS with smoothing

그러나 greedy TS with smoothing도 target leakage가 발생한다. Target leakage는 예측 시점에서 사용할 수 없는 변수가 데이터셋에 포함되는 문제를 말한다. Train set에 사용될 $\hat{x}_k^i$가 타겟변수를 통해 만들어 짐으로써 target leakage가 생기는 것이다. 극단적인 예시를 들어서 설명하고자 한다.
$i-th$ feature는 모두 다른 값을 가진(unique) categorical 변수라고 가정하자. 또한 $P(y=1|x^i) = 0.5, x^i \in \{A, B, C, D, E, F, \dots.\}$이다. 먼저 train data는 $x^i \in \{A, B, C, D, E, F\}$라고 하면 이때 TS는 아래와 같이 구해진다. 만약 split criterion이 $x^i = \frac{0.5+ap}{1+a}$ 로 설정되면 training data는 완벽하게 분류된다.

		x^i		y
I_1	…	A	…	1
I_2	…	B	…	1
I_3	…	C	…	1
I_4	…	D	…	0
I_5	…	E	…	0
I_6	…	F	…	0

		x^i(TS)		y
I_1	…	$\frac{1+ap}{1+a}$	…	1
I_2	…	$\frac{1+ap}{1+a}$	…	1
I_3	…	$\frac{1+ap}{1+a}$	…	1
I_4	…	$\frac{0+ap}{1+a}$	…	0
I_5	…	$\frac{0+ap}{1+a}$	…	0
I_6	…	$\frac{0+ap}{1+a}$	…	0

표 5 calculation of target statistic in train datasets

하지만 test data에는 split criterion이 적용될 수 없다. 만약 test data가 $x^i \in \{G, H, I, J\}$ 라면 TS 계산은 아래와 같다. All feature value of test instance는 p이므로, split criterion을 가지고 test data를 구분할 수 없게 된다.

		x^i		y
I_1	…	A	…	1
I_2	…	B	…	1
I_3	…	C	…	1
I_4	…	D	…	0
I_5	…	E	…	0
I_6	…	F	…	0
I_7		G		1
I_8		H		1
I_9		I		0
I_{10}		J		0

		x^i(TS)		y
I_1	…	$\frac{1+ap}{1+a}$	…	1
I_2	…	$\frac{1+ap}{1+a}$	…	1
I_3	…	$\frac{1+ap}{1+a}$	…	1
I_4	…	$\frac{0+ap}{1+a}$	…	0
I_5	…	$\frac{0+ap}{1+a}$	…	0
I_6	…	$\frac{0+ap}{1+a}$	…	0
I_7	…	$\frac{0+ap}{0+a} = p$	…	1
I_8	…	p	…	1
I_9	…	p	…	0
I_{10}	…	p	…	0

표 6 calculation of target statistic in test datasets

즉, train set과 test set에 따라 $\hat{x}^i|y$ 의 분포가 달라지는 conditional shift 문제가 발생할 수 있다. 논문에서는 conditional shift 문제를 방지하기 위해 TS에 대한 property들을 정의하였다. 첫번째 property(P1)는 다음과 같다.

$$P1\ \ \mathbb{E}(\hat{x}^i \mid y = v) = \mathbb{E}(\hat{x}_k^i \mid y_k = v)\ \ where\ (x_k, y_k) is\ the\ k-th\ training\ example$$

$y = v$ 로 주어졌을 때 테스트데이터의 expectation과 $y_k = v$ 로 주어졌을 때 학습데이터의 expectation이 같아야 한다. 앞선 우리의 예제에서는 $\mathbb{E}(\widehat{x_k^i}|y_k) = \frac{y_k + ap}{1+a}$와 $\mathbb{E}(\hat{x}^i|y) = p$ 로 서로 달랐다. 따라서 Greedy TS의 경우 이 성질을 만족하지 못한다.
두번째 property(P2)는 다음과 같다. 모델을 학습하고 TS를 계산하는 과정에서 모든 트레인 데이터가 효율적으로 사용되어야 함을 의미한다.

$$P2\ Effective\ usage\ of\ all\ training\ data\ for\ calculating\ TS\ features\ and\ for\ learning\ a\ model.$$

Holdout TS

Conditional shift 문제를 피하기 위한 genal idea는 k번째 training example의 TS를 계산할 때 x_k를 제외한 subset of examples $\mathcal{D}_k \subset \mathcal{D} \setminus \{x_k\}$에 대해 TS를 계산하는 것이다. x_k를 빼고$\mathcal{D}_k$ 를 사용하면 현재 feature value를 계산하고자 하는 객체에 대한 y_k 값은 사용되지 않는다. 식으로 표현하면 다음과 같다.

$$\hat{x}_k^i = \frac{\sum_{x_j \in \mathcal{D}_k} 1_{\{x_j^i = x_k^i\}} \cdot y_j + ap}{\sum_{x_j \in \mathcal{D}_k} 1_{\{x_j^i = x_k^i\}} + a}$$

k번째 example의 TS를 x_k를 제외하고 계산하는 방법 중 하나로, Training dataset을 $\mathcal{D} = \hat{\mathcal{D}}_0 \sqcup \hat{\mathcal{D}}_1$ 2개의 파트로 나눈 후 $\hat{\mathcal{D}}_0$은 TS 계산에 사용하고 계산된 TS를 바탕으로 $\hat{D}_1$를 학습에 사용할 수 있다. 이 방법은 P1은 만족하지만 모델링과 TS를 계산하는 데에 사용하는 데이터의 양을 감소시키므로 주어진 데이터를 효율적으로 사용할 수 없다. 즉, 또 다른 TS의 성질인 P2를 위반하게 된다.

$$P2\ Effective\ usage\ of\ all\ training\ data\ for\ calculating\ TS\ features\ and\ for\ learning\ a\ model.$$

Leave-one-out TS

$\mathcal{D}_k = \mathcal{D} \setminus x_k$를 training set의 TS를 만드는 데에 사용하고, 모든 dataset인 $\mathcal{D}_k = \mathcal{D}$를 test set의 TS를 만드는 데에 사용하는 leave-one-out 방법은 $P2$를 만족한다. 하지만 이 방법은 target leakage 문제가 발생할 수 있고, 이에 따라 training set은 잘 예측하지만 test set은 그러지 못할 가능성이 있다. 아래 예시를 통해 확인해보자.

		x^i		y
I_1	…	A	…	1
I_2	…	A	…	0
I_3	…	A	…	1
I_4	…	A	…	0
I_5	…	A	…	1

		x^i		y
I_1	…	$\frac{2+ap}{4+a}$	…	1
I_2	…	$\frac{3+ap}{4+a}$	…	0
I_3	…	$\frac{2+ap}{4+a}$	…	1
I_4	…	$\frac{3+ap}{4+a}$	…	0
I_5	…	$\frac{2+ap}{4+a}$	…	1

표 7 leave one out TS

cut off value가 $\frac{2.5+ap}{4+a}$라면, training dataset은 완벽하게 분류된다. 변별력이 높은 것처럼 보이나, test data의 x^i에 대한 TS를 계산하면 항상 $\frac{3+ap}{5+a}$로 모든 객체들이 동일한 값을 갖고 split을 할 수 없으므로 변별력이 없다.

Ordered TS

CatBoost에서는 ordering principle을 이용하여 TS를 계산한다. 먼저 실제 우리의 데이터가 시계열 데이터가 아닐지라도 가상의 시간을 도입한다. Tree를 만들 때 마다 (학습을 진행할 때마다) permutation을 시행하여 각 데이터에 순서를 부여한다. 이 순서가 가상의 시간이 되는 것이다.

Training example에 random permutation σ를 해당 데이터의 time이라고 임의적으로 지정하고, 이를 이용하여 데이터의 observed history 값만 이용해서 TS를 계산한다. 앞에서 언급했던 TS를 구하는 식 $\hat{x}_k^i = \frac{\sum_{x_j \in \mathcal{D}_k} 1_{\{x_j^i = x_k^i\}} \cdot y_j + ap}{\sum_{x_j \in \mathcal{D}_k} 1_{\{x_j^i = x_k^i\}} + a}$에서 training example은 $\mathcal{D}_k = \{x_j : \sigma(j) < \sigma(k)\}$, test sample은 $\mathcal{D}_k = \mathcal{D}$를 이용하여 history data들을 사용한 TS를 구할 수 있다.

밑의 그림 예시와 같이, $\sigma = 6$인 x_k의 TS를 구할 때 $\sigma = 1 \sim 5$까지의 값만 사용되는 것이다. 이러한 방식은 이전 시점의 데이터를 이용하여 target leakage가 일어나지 않는다는 점에서 P1를 만족하고, TS 계산 및 모델 학습에 training example을 효율적으로 사용한다는 점에서 P2 또한 만족한다.

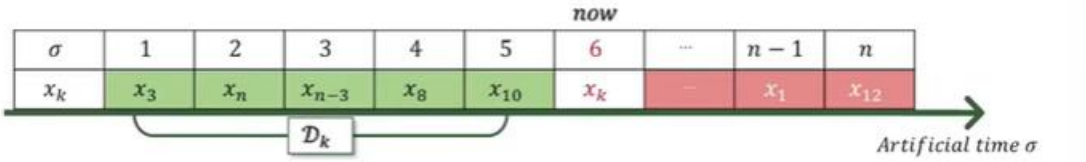

그림 2 ordered TS 예시

실제 ordered TS 계산은 다음과 같다.

		x^i		TS	y
I_1	…	A	…	$\frac{0+0.1\times 0}{0+0.1}=0$	1
I_2	…	B	…	$\frac{0+0.1\times 1.0}{0+0.1}=1.0$	1
I_3	…	C	…	$\frac{0+0.1\times 1.0}{0+0.1}=1.0$	1
I_4	…	A	…	$\frac{1+0.1\times 1.0}{1+0.1}=1.0$	0
I_5	…	B	…	$\frac{1+0.1\times 0.75}{1+0.1}=0.977$	1
I_6	…	C	…	$\frac{1+0.1\times 0.8}{1+0.1}=0.982$	1
I_7	…	B	…	$\frac{2+0.1\times 0.833}{2+0.1}=0.992$	0
I_8	…	C	…	$\frac{2+0.1\times 0.714}{2+0.1}=0.986$	1
I_9	…	C	…	$\frac{2+0.1\times 0.75}{3+0.1}=0.992$	1
I_{10}	…	C	…	$\frac{3+0.1\times 0.667}{4+0.1}=0.748$	0

표 8 ordered TS 실제 계산 방법 예시

4. Prediction shift and ordered boosting

4.1 Prediction shift

Catboost에서는 target leakage 문제로 인해 예측결과가 변하는 prediction shift 문제가 존재한다. 이를 ordered boosting이라는 개념을 통해 해결하고자 한다. 이는 ordered TS method와 유사하다. 앞서 Gradient boosting에서 h^t 는 다음과 같이 expectation을 통해 표현되었다.

$$h^t = argmin_{h\in H}\mathbb{E}\mathcal{L}\big(y, F^{t-1}(x)+h(x)\big) = \underset{h\in H}{\operatorname{argmin}}\,\mathbb{E}\big(-g^t(x_k,y_k)-h(x_k)\big)^2$$

실제로 expectation은 알 수 없기 때문에 현재 관찰가능한 dataset $\mathcal{D}$를 이용하여 다음과 같은 식으로 근사하여 계산된다.

$$h^t = \underset{h \in H}{\operatorname{argmin}} \frac{1}{n} \sum_{k=1}^{n} \left(-g^t(x_k, y_k) - h(x_k)\right)^2$$

이로 인해 다음과 같이 연쇄적으로 prediction의 shift가 발생하게 된다.

1. train의 데이터셋에 대한 gradient의 the conditional distribution $g^t(x_k, y_k)|x_k$은 distribution on a test example $g^t(x, y)|x$으로부터 shift된다.
2. 조건부 확률이 다르기 때문에 h^t 는 solution에서 biased가 생긴다.
3. 결국 이것은 trained model $F^t(= F^{t-1} + \alpha h^t)$의 generalization ability에 영향을 미친다.

이와 같이 각각의 step에서 사용되는 gradient들은 현재 모델F^{t-1}을 만드는데 쓰이는 data point들의 target value를 사용하여 추정된다. 즉 TS관점에서 이러한 문제는 target leakage를 발생시킨다.

Prediction shift 문제를 해결하기 위해 부스팅의 매 스텝마다 independent한 새로운 dataset을 사용하면 unshifted residual을 얻을 수 있을 것이다. 이러한 원리에 따르면 i번째 트리를 학습하는 상태라고 할 때, (x_k, y_k)를 넣었을 때의 residual이 shift하지 않으려면 $i-1$번째 트리를 학습할 때까지 x_k가 사용되지 않아야 한다. 그러나 실제로는 매 트리에 다른 데이터를 사용할 만큼 데이터가 무한한 경우가 거의 없다. 이를 해결하기 위해 CatBoost에서는 ordering principle을 모델에 적용한다. Training example에 permutation σ를 random하게 지정하고, permutation 순서 상 처음 i개 examples만을 이용하여 학습하는 모델 $M_1, \dots, M_i, \dots, M_n$을 만든다. j번째 sample에 대한 residual을 계산하는 과정에서 모델 M_{j-1}을 사용하여 shift를 피할 수 있다. 해당 과정을 그림으로 표현하면 밑의 그림과 같다.

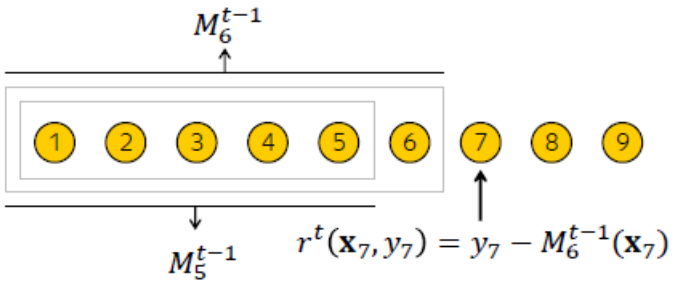

그림 3 ordered boosting 예시1

그림3에서 x_7의 residual을 구하기 위해 사용되는 모델 M_6^{t-1}은 7번 example이 학습에 사용되지 않은 데이터이므로 shift 문제를 피할 수 있다. 그러나 ordering boosting은 사용 데이터가 다른 n개의 모델을 학습해야 한다는 점에서 모델의 복잡성이 높아진다는 문제점이 있다.

위 그림을 수식으로 표현하면 아래 알고리즘과 같다.

Algorithm 1: Ordered boosting

input : $\{(\mathbf{x}_k, y_k)\}_{k=1}^{n}, I;$

$\sigma \leftarrow$ random permutation of $[1, n]$;

$M_i \leftarrow 0$ for $i = 1..n;$

for $t \leftarrow 1$ **to** I **do**

  for $i \leftarrow 1$ **to** n **do**

    $r_i \leftarrow y_i - M_{\sigma(i)-1}(\mathbf{x}_i);$

  for $i \leftarrow 1$ **to** n **do**

    $\Delta M \leftarrow LearnModel((\mathbf{x}_j, r_j) : \sigma(j) \leq i);$

    $M_i \leftarrow M_i + \Delta M$;

return M_n

알고리즘 2 ordered boosting algorithm

앞서 categorical features를 처리하기 위해 본 논문에서 제안한 ordered TS 또한 변수의 각 category에 permutation을 설정하는데, ordered TS와 Ordered boosting을 같은 값으로 지정하여 두 방법을 결합한다.

5. Practical Implementation of Ordered Boosting

CatBoost에는 일반적인 gradient boosting에 ordered TS를 계산하는 알고리즘이 포함되어 있는 plain mode와, ordered boosting과 ordered TS를 사용하는 ordered mode가 있다. 그 중 ordered mode는 training set에 대해 s+1개의 random permutation 값을 생성한다. 이후 $\sigma_1, \ldots, \sigma_s$ 는 트리 구조를 정의하는 분할에 사용되고, σ_0 은 만들어진 트리의 leaf value를 지정하는 데에 사용된다. 이 때 permutation을 한 번 진행하는 것은 prediction value의 variance를 높일 수 있으므로 이를 여러 번 시행하여 variance가 감소하도록 한다.

Building a tree

CatBoost의 predictor들은 oblivious decision tree 형식을 갖는데, 이 때 oblivious는 모든 트리 레벨에서 같은 분할 기준이 사용되는 것을 의미한다. 이러한 tree는 balanced tree이며 overfitting에 강하고 testing time을 단축시킨다는 장점이 있다.

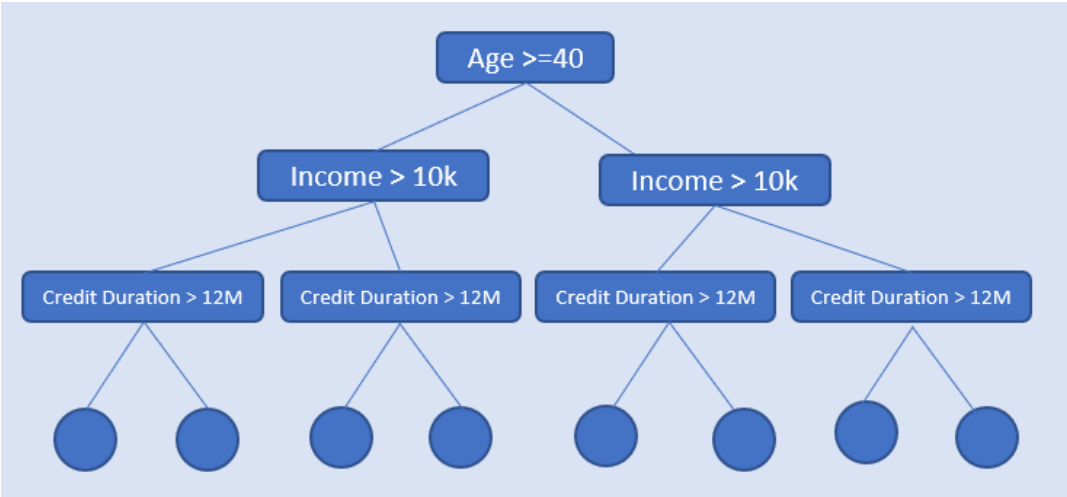

그림 4 oblivious tree 예시

Tree를 생성하는 과정은 전체적인 알고리즘은 아래와 같다.

Algorithm 2: Building a tree in CatBoost

input : M, $\{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$, α, L, $\{\sigma_i\}_{i=1}^{s}$, $Mode$

$grad \leftarrow CalcGradient(L, M, y)$;
$r \leftarrow random(1, s)$;
if $Mode = Plain$ **then**
  $G \leftarrow (grad_r(i)$ for $i = 1..n)$;
if $Mode = Ordered$ **then**
  $G \leftarrow (grad_{r,\sigma_r(i)-1}(i)$ for $i = 1..n)$;
$T \leftarrow$ empty tree;
foreach *step of top-down procedure* **do**
  foreach *candidate split* c **do**
    $T_c \leftarrow$ add split c to T;
    if $Mode = Plain$ **then**
      $\Delta(i) \leftarrow \mathrm{avg}(grad_r(p)$ for
      $p: \ leaf_r(p) = leaf_r(i))$ for $i = 1..n$;
    if $Mode = Ordered$ **then**
      $\Delta(i) \leftarrow \mathrm{avg}(grad_{r,\sigma_r(i)-1}(p)$ for
      $p: \ leaf_r(p) = leaf_r(i), \sigma_r(p) < \sigma_r(i))$
      for $i = 1..n$;
    $loss(T_c) \leftarrow \cos(\Delta, G)$
  $T \leftarrow \arg\min_{T_c}(loss(T_c))$
if $Mode = Plain$ **then**
  $M_{r'}(i) \leftarrow M_{r'}(i) - \alpha\,\mathrm{avg}(grad_{r'}(p)$ for
  $p: \ leaf_{r'}(p) = leaf_{r'}(i))$ for $r' = 1..s, i = 1..n$;
if $Mode = Ordered$ **then**
  $M_{r',j}(i) \leftarrow M_{r',j}(i) - \alpha\,\mathrm{avg}(grad_{r',j}(p)$ for
  $p: \ leaf_{r'}(p) = leaf_{r'}(i), \sigma_{r'}(p) \leq j)$ for $r' = 1..s$,
  $i = 1..n, j \geq \sigma_{r'}(i) - 1$;
return T, M

알고리즘 3 building a tree in Catboost

알고리즘을 단계에 대한 세부 설명은 다음과 같다.

	설명
$r \leftarrow random(1, s)$	순열의 위치에 따라 prediction 분산에 차이가 생기므로 이를 방지하기 위해각 iteration 마다 $\{\sigma_1, \ldots, \sigma_s\}$ 중 sampling. - 이를 기반으로 tree T_t를 만든다. - 또한, 모든 TS는 이 permutation을 통해 계산된다.
Model $M_{r,j}$	순열σ_r에서 앞 j 개의 instance를 사용하여 Training한 현재의 model
$M_{r,j}(i)$	$M_{r,j}$ 의 i번째 instance에 대한 예측값 (the current prediction for the i-th example based on the first j examples in the permutation σ_r)
$grad_{r,j}\ (i)$	$\frac{\partial L(y_i, s)}{\partial s}\ \vert_{s=M_{r,j}(i)}$
$grad_{r,\sigma(i)-1}(i)$	순열 σ_r에서 instance i의 앞에 위치한 instance들로 training한 model $M_{r,\sigma_r(i)-1}$의 gradient
$loss(T_c) \leftarrow \cos\ (\Delta, G)$	분할 후보c에 대한 loss는 leaf value 와 각 instance의 gradient의 cosine similarity로 계산된다. (While constructing a tree, the loss of the candidate split c is computed by the cosine similarity between the leaf value and the gradient of each example.) Cosine similarity $similarity(A, B) = \frac{A \cdot B}{\vert\vert A\vert\vert \times \vert\vert B\vert\vert} = \frac{\sum_{i=1}^{n} A_i \times B_i}{\sqrt{\sum_{i=1}^{n} A^2}\sqrt{\sum_{i=1}^{n} B^2}}$
$leaf_r(i)$	순열σ_r에 따라 달라지는 instance i 의 leaf node
$leaf\ value\ \Delta(i)$	순열 σ_r 에서 instance i 보다 앞에 위치하면서 동시에 $leaf_r(i)$ 에 속하는 instance p들의 gradient인 $grad_{r,\sigma(i)-1}(i)$의 평균

Choosing leaf values

학습이 다 완료된 후 최종적인 leaf node를 계산하는 과정이다. permutation을 $s+1$번 진행하였고 이때 1번 남겨놓은 permutation을 바탕으로 gradient boosting 절차에 의해 final model F의 leaf value가 계산된다. training example i는 $leaf_o(i)$에 대응된다. 최종 모델 F에 대하여 test를 할 때 ordered TS는 모든 training data를 사용하여 계산된다. 즉, test에서는 일반적인 TS를 계산하는 것과 동일하다.

Feature combinations

catboost의 또 다른 중요한 점은 categorical features들의 combination을 사용한다는 점이다. 광고 클릭률 예측 과제를 예를 들면, 광고 토픽과 user ID의 joint information이 고차원의 의존성을 가질 수 있다. 물론 가능한 combination의 수는 dataset에서 categorical features의 수에 따라 기하급수적으로 증가하기 때문에 모든 조합을 처리하는 것은 불가능하다. Catboost는 greedy한 방법으로 조합을 구성한다. 즉, 현재 tree에서 "이전 split에 사용되었던 모든 categorical features"과 "데이터셋 내의 모든 categorical features"를 combine한다. 물론 이렇게 만들어진 combination들은 즉시 TS로 전환된다.

CatBoost 실습

함수 설명

python에서 CatBoost는 회귀문제에는 CatBoostRegressor 함수를 이용하며, 분류문제에서는 CatBoostClassifier를 사용한다. R에서 CatBoost는 회귀문제와 분류문제 모두 catboost.train 함수를 이용한다. CatBoost에서는 함수 사용 시 조정할 수 있는 파라미터의 이름이 R과 python모두 동일하다. 각 parmeter에 대한 설명을 아래와 같다.

Parameter	설명
loss_function[default='RMSE']	학습에 사용할 평가 지표. 회귀 – RMSE, MAE, MAPE 등 / 분류 – Logloss, CrossEntropy 등
iterations[default=1000]	최대로 학습할 트리 개수 Iteration이 감소하면 learning rate은 증가해야만 한다.
learning_rate [default=loss function이 RMSE일 경우 iterations 횟수에 따라 자동으로 지정]	이전 트리 정보에 대한 학습률 만약 overfitting이 보이면 learning rate을 줄여야 한다. 만약 overfitting이 없고 마지막 iteration에서도 evaluation datasets error가 여전히 줄어든다면 learning rate을 올려야 한다.
l2_leaf_reg[default=3]	L2 정규화 파라미터
Subsample [default=1 if datasets with less than 100 objects]	매 단계마다 사용할 샘플의 비율
depth[default=6]	트리의 깊이
min_data_in_leaf[default=1]	각 leaf 내에 속하는 최소 샘플 개수
max_leaves[[default=31]	최대 leaf 개수
one_hot_max_size[default=NA for CPU]	범주형 변수 내 category가 이 값보다 작을 경우 one-hot encoding을 사용함.
rsm[default=1]	각각의 split selection 단계에서 변수 사용 비율
nan_mode[default="Min"]	Missing value 처리방법. "Forbidden": 에러 처리/ "Min": 해당 변수의 최소값 사용" / "Max": 해당 변수의 최대값 사용
boosting_type[default="Plain" for CPU]	부스팅 방법. "Plain": 기본적인 gradient boosting / "Ordered": Ordered boosting 방법 이용
border_count[default=254 for CPU]	한 변수가 분할 가능한 횟수.